Feature-Pair Refactor of the Polygon Force Kernels

(biarc target; normal and single-arc rounded as stepping stones)

1 Scope and terminology

The physical target model is the **biarc** polygon: each vertex has two tangent-continuous circular arcs meeting at a junction j on the inward bisector (§4 of *packing_treatise-7*). Polygon s with n_s vertices therefore has

- n_s straight edges (one per consecutive vertex pair), and
- $2n_s$ arcs (two per vertex: "arc 1" from a_i^- to the junction j_i, "arc 2" from j_i to a_i^+).

So $3n_s$ features total per polygon.

The codebase currently has a *single-arc rounded* kernel (`updateForceEnergyRoundedKernel`) that approximates biarcs by a single arc per vertex — $2n_s$ features (n_s edges + n_s arcs). That kernel is FD-validated and useful, but it is a stepping stone. The *normal* model uses only edges (n_s features per polygon) and is structurally closest to what we want to refactor first.

We will build the refactor parametric in the feature set per vertex, so the same Phase A/B/C scaffolding services all three models. The biarc model is the destination; we walk to it via normal → single-arc rounded → biarc.

2 Problem statement

The current single-arc rounded force/energy kernel (`updateForceEnergyRoundedKernel`) uses per-vertex parallelism, but each thread performs $O(n)$ sequential work inside the kernel: it loads polygon B's full geometry, computes B's per-vertex arc geometry, walks all of B's edges/arcs to find crossings with the thread's local features, then walks crossings in order. With $n = 32$, this serializes a 32× factor inside every thread before any of the GPU parallelism is brought to bear. The biarc kernel would inherit and amplify this (3× more features per vertex) if we built it the same way.

Worse, because each thread does its own "does my feature go inside B?" determination via B's local geometry, an interior vertex deep in B (far from any edge crossing) only learns it is inside if the ball list reaches a B-vertex from that interior point. That forces the search radius up to ∼polygon_radius, which is independent of the vertex-pair interaction range. The result is a slow kernel and a fragile, polygon-size-dependent search radius.

The same Green's-theorem accounting can be done much more cleanly — and the normal model already does most of it — by reformulating around *feature pairs* and a global sorted intersection list. This document specifies the refactor.

3 Architecture overview

Replace each force/energy call (normal, rounded, areaSquared) with three phases of small per-thread kernels. Every kernel is per-pair or per-feature, with $O(1)$ work per thread aside from inner loops bounded by physics (number of crossings on a single feature, typically ≤ 4).

- **Phase A — Build.** For each vertex-vertex candidate pair (i_a, i_b) in the existing per-vertex ball list, emit the feature-pair candidates the chosen model requires (Section 4).
- **Phase B — Probe.** One thread per feature-pair candidate. Compute pairwise intersection. If there is a crossing, emit the crossing record. Add the per-crossing ("external") atomic-add force contributions to the four endpoint vertices and the partner-side coupling (Section 5).
- **Phase C — Assemble.** Sort the global crossing list, then one thread per A-feature walks its sorted crossings to determine the inside/outside chord segments and contribute the per-segment ("internal") Green's integral terms to energy and to the chord endpoints (Section 6).

Two consequences worth highlighting up front:

Search radius returns to $2L$. Because the inside/outside state of each A-feature is recovered from the sorted crossing list in Phase C — not from each thread independently probing B's geometry — only the crossings themselves need to be discovered. A crossing of two convex edges of lengths at most L puts the four endpoint vertices within $2L$ of each other, so the per-vertex ball reach can be $\sim 2.5L$ and that suffices. The polygon-size dependence in the search radius disappears.

No `processedShapes` cap. The current kernel dedups neighbor shapes per thread and caps unique shapes at 32; the cap was a performance band-aid that also subtly biased results when more shapes existed than fit. After the refactor we enumerate *feature pairs*, not neighbor shapes, and there is no fan-out cap.

4 Feature representation

A *feature* is one segment of a polygon's boundary. We support three feature types and bind each to a vertex index i:

- `EDGE` at i: the straight segment between two consecutive tangent points. For the normal model, this is just the segment from vertex i to vertex $i+1$. Used by all three models.
- `ARC1` at i: the first arc at vertex i, from a_i^- to the junction j_i. Used by biarc only. For the single-arc rounded model we collapse `ARC1` and `ARC2` into one feature labeled `ARC1` (the single arc) and never emit `ARC2`.
- `ARC2` at i: the second arc, from j_i to a_i^+. Used by biarc only.

We encode a feature ID in a single `uint32_t` with the type in the low two bits:

$$\text{featureId}(s, i, t) \;=\; 4 \cdot (\text{startIndices}[s] + i) + t, \qquad t \in \{\text{EDGE} = 0,\ \text{ARC1} = 1,\ \text{ARC2} = 2\}.$$

The global vertex index is $\lfloor \text{featureId}/4 \rfloor$, the type is `featureId & 3`, and `shapeId[]` gives the owning polygon.

Feature counts per polygon by model:

Model	features per vertex	features per polygon
normal	1 (`EDGE`)	n
single-arc rounded	2 (`EDGE`, `ARC1`)	$2n$
biarc	3 (`EDGE`, `ARC1`, `ARC2`)	$3n$

5 Phase A: candidate generation

Input. The per-vertex ball list (`ballNeighbors[]`, `numBallNeighbors[]`) already populated by either `updateNeighborCells()` or `updateNeighborBall()`. This phase reuses Decision 1's choice — the vertex-vertex list is the source of truth for spatial proximity.

Output. A flat candidate-pair array `featurePairs[]` where each entry is

```
struct FeaturePair { uint32_t fA; uint32_t fB; };
```

plus a count.

Logic. One thread per entry of the per-vertex ball list. For the pair (i_a, i_b) that thread is handling, it emits, depending on the active model:

For each vertex-vertex candidate (i_a, i_b) we emit the Cartesian product of i_a's feature set with i_b's feature set:

Model	pairs per (i_a, i_b)	types emitted
normal	$1 \times 1 = 1$	edge×edge
single-arc rounded	$2 \times 2 = 4$	all combinations of {edge, arc1}
biarc	$3 \times 3 = 9$	all combinations of {edge, arc1, arc2}

Per the user's direction ("assume the worst"), we emit *all* pairs unconditionally, including the (arc, arc) combinations where a crossing is rare. We accept the extra memory in exchange for not having to reason about edge-case filtering.

The candidate count grows by a factor of 9 (biarc) over the vertex-vertex list. With $N \cdot n = 2048$ vertices and a typical 575 candidates per vertex (Section 10), this is roughly $9 \cdot 2048 \cdot 575/2 \approx 5.3 \times 10^6$ pairs at peak. Stored as a pair of `uint32_t` per entry, that's ∼42 MB at peak in the worst case (pre-relax, all-to-all); post-relax it drops by ∼200×.

The emission is parallel and produces output to consecutive locations. We allocate the output array with a worst-case bound ($4 \cdot \text{ballMaxNeighbors} \cdot \text{size}$) and use a single `atomicAdd` on a shared counter to claim slots, or use CUB's `DeviceScan` to compact in a second pass — whichever is simpler. Resizing on overflow follows the existing pattern.

6 Phase B: pairwise probe

Input. `featurePairs[]` from Phase A.

Output.

- A flat array of crossing records (one per crossing, with A-feature ID and parameter for sorting).
- Direct `atomicAdd`-style force contributions for the "external" per-crossing terms.

```
struct Crossing {
    uint32_t fA;          // owning A-feature
    uint32_t fB;          // partner B-feature
    float    paramA;      // parameter along fA at crossing (for sorting)
```

```
    double   Xx, Xy;      // crossing point
    double   eBx, eBy;    // tangent direction of fB at crossing (for forces)
    double   paramB;      // parameter along fB at crossing
};
```

Kernel. One thread per entry of `featurePairs[]`. The thread reads the pair (f_a, f_b), decodes their types (low bit), loads the four (or six — for arcs, plus centers and angles) coordinates needed, and dispatches to the appropriate pairwise intersection routine:

- `edgeEdgeCross` — two line-segments. 0 or 1 crossing. (Already exists in the codebase as part of `updateNeighborsKernel`; factor it into `intersect.cuh`.)
- `edgeArcCross` — segment intersect circle, gated to the arc parameter range and the edge range. 0, 1, or 2 crossings. Works identically for `ARC1` and `ARC2`; the only per-arc data passed in are center, radius, and angle range.
- `arcEdgeCross` — same as above with the roles swapped.
- `arcArcCross` — two circles. 0, 1, or 2 crossings. Also agnostic to whether each side is `ARC1` or `ARC2`.

The dispatch in Phase B's kernel decodes the type bits of the two feature IDs to pick the right routine, and reads the appropriate center/radius/range data for each arc type. Biarc adds no new intersection math — just two more arc variants on each side.

Each crossing is appended to the output array (atomic counter). For each crossing, the thread also applies the external force contributions: the chain-rule force from the implicit function theorem (dX_c/dv contributions on the four endpoint vertices and the partner-feature coupling on B's side). These are local to the crossing and do not depend on inside/outside state, so they fire here.

No inner loops over n anywhere in this phase.

7 Phase C: assembly

Step 1 — global sort. Sort the crossing records by `(fA, paramA)` using CUB's `DeviceRadixSort::SortPairs`, the same pattern `updateOutersections` already uses for the normal model. The packed sort key can be `(fA << 32) | floatBits(paramA)` into a `uint64_t`, then a permutation applied to the full record array.

Step 2 — per-feature walk. One thread per A-feature reads its contiguous slice of the sorted list, sorts the crossings by `paramA` (at most $\sim$4 entries; a small insertion sort), and walks them in order:

1. Determine the inside/outside state at the start of the feature. For `EDGE` at vertex i, this is "is vertex i inside any of the polygons whose features cross this edge?" Equivalently: walk the polygon-pair list of this A-feature, count crossings up to the start parameter, and take parity. *Implementation note:* since each polygon B that crosses f_a contributes an even number of crossings on f_a when f_a enters/exits B completely, the parity at the start can be deduced from how many B-polygons contain f_a's start endpoint. This is the same "which shapes contain me" logic the normal model already implements.
2. For each consecutive pair of crossings (and the segments before the first and after the last), determine whether that sub-segment of f_a is inside the relevant other polygon, and if so, contribute the internal Green's-theorem term for that sub-segment to:

- the global energy (overlap area term), and
- the forces on the segment's endpoint vertices (the chord-line-integral force pieces).

For `ARC` features the integration is over a circular arc rather than a straight segment; the per-segment integrand is the standard arc-area-with-respect-to-origin formula. Edge features use the straight-line variant.

No inner loops over n anywhere in this phase either — the walk is over the feature's own crossings, bounded by physics.

8 Shared helpers

Per Decision 5, factor out the routines that are used across normal, rounded, and areaSquared:

- `src/features.cuh` — Feature ID encode/decode, per-feature geometry accessors (edge endpoints, arc center + radius + angle range, all derivable from `positions[]` and `shapeId/startIndices/next/pre`
- `src/intersect.cuh` — the four pairwise intersection routines (`edgeEdgeCross`, `edgeArcCross`, `arcEdgeCross`, `arcArcCross`). Each returns its crossings as a small fixed-size buffer.
- `src/integrate.cuh` — per-segment Green's contribution for an edge chord and an arc chord, with respect to a per-pair anchor. Force gradients of these terms.
- `src/parityWalk.cuh` — the "walk sorted crossings and figure out which shapes contain me" helper. Used identically by the normal model's existing flow and by Phase C above.
- `src/forces/normal.cu` — normal model entry point: Phase A emits edge×edge only, then B and C.
- `src/forces/rounded.cu` — single-arc rounded entry point: Phase A emits the 4 {edge, arc1} combinations.
- `src/forces/biarc.cu` — biarc entry point: Phase A emits the 9 {edge, arc1, arc2} combinations.
- `src/forces/areaSquared.cu` — wraps biarc (or rounded) with the existing 2-pass area-scaling.

9 Implementation order

Following Decision 5: start with the normal model. Then walk up the feature-set ladder.

1. **Phase 0 — shared scaffolding (no behavior change).** Create the headers above with the encode/decode and helper signatures; move the existing line-line intersection logic out of `updateNeighborsKernel` into `intersect.cuh`. Verify normal model still passes existing FD tests.
2. **Phase 1 — normal model on new architecture.** Replace `updateNeighbors / updateOutersections / updateForceEnergyExterior / updateForceEnergyInterior / updateForceEnergyVertexDisk` with the new A/B/C flow. The existing logic is conceptually already this shape, so this step is mostly restructuring + factoring shared helpers. Phase A emits only edge×edge. Validate with the existing normal-model FD tests: force $= -\nabla E$ to the existing 0.1% tolerance.

3. **Phase 2 — single-arc rounded model on new architecture.** Add the line-arc and arc-arc routines to `intersect.cuh` and the arc-segment Green's terms to `integrate.cuh`. Phase A emits the 4 combinations of {edge, arc1}. Build `forces/rounded.cu`. Validate with the existing rounded FD tests: force $= -\nabla E$ to the existing 1.6×10^{-5} tolerance.

4. **Phase 3 — biarc model.** Add `ARC2` as a feature type and the biarc-junction geometry (radii d_1, d_2, centers z_1, z_2, angle ranges from §4 of *packing_treatise-7*). Phase A emits the 9 combinations of {edge, arc1, arc2}. Phase B's intersection dispatch already covers (arc, arc) and (arc, edge); no new math, only the geometry-construction helper needs an `arc2` variant. Phase C's arc-segment Green's term applies unchanged. FD-validate against the same target tolerance as rounded.

5. **Phase 4 — areaSquared.** Two-pass mode on top of biarc (or rounded, depending on what's wanted). Validate with existing areaSquared tests.

6. **Phase 5 — delete the old code paths.** See Section 9.

Each phase is independently validated. The old per-vertex `updateForceEnergyRoundedKernel` stays compileable in the source tree until Phase 4 so we have a reference to FD against if Phase 2 introduces a regression.

10 What we delete after Phase 4

Per Decision 6 (and confirmed: anything not compatible):

- `updateForceEnergyRoundedKernel` (the 1500-line monolith).
- `updateForceEnergyAreaSquaredCUDA` (replaced by the new 2-pass wrapper).
- The per-thread `roundedScratch` buffer of $38n$ doubles per vertex.
- The `processedShapes[32]` dedup logic and the `maxProc` cap.
- The polygon-aware `searchFactor` default in `setStartIndices` (revert to fixed 2.5).
- The polygon-aware `trueCutoff` in `updateNeighborBall` (revert to $2 \cdot$ maxEdgeLength).
- The `updateForceEnergyExteriorKernel` and `updateForceEnergyInteriorKernel` (their work is absorbed into Phase B and Phase C respectively).
- `updateForceEnergyVertexDiskKernel` and its ball variant — the vertex-disk cap is just a special case of edge-edge / arc-edge crossings in the new flow.
- The compatibility scaffolding around two neighborTypes can stay; both `updateNeighborCells` and `updateNeighborBall` keep populating the per-vertex candidate buffer that Phase A reads.

11 Memory and performance estimates

For the N=64, n=32, ϕ=1.1 working case:

Total vertices (Nn):	2048
Candidates per vertex (post-relax, ball = $2.5L$):	$\sim$12
Candidates per vertex (pre-relax, all-to-all):	2016
Feature-pairs (normal, peak):	$2048 \cdot 2016/2 \approx 2 \times 10^6$
Feature-pairs (rounded, peak):	$4 \cdot 2 \times 10^6 = 8 \times 10^6$
Feature-pairs (biarc, peak):	$9 \cdot 2 \times 10^6 \approx 1.8 \times 10^7$
Feature-pair memory at biarc peak:	$\sim$144 MB (one shot, pre-relax)
Feature-pair memory post-relax:	$\sim$0.7 MB
Crossings emitted (rough estimate):	$\sim 10^5$
Crossings memory:	$\sim$5 MB

After softBody relaxation (when polygons are at target shapes and ball list is sparse), the feature-pair count drops by $\sim 200\times$ to $\sim$40K, well within memory budgets and dominated by Phase B's $O(1)$-per-thread work.

12 Open items for later

- Containment cases (one polygon fully inside another with no edge crossings) emit zero crossings from Phase B. Phase C handles this correctly *if* we seed it with a polygon-pair "A's start endpoint inside B?" flag. Decide whether that flag comes from a separate cheap kernel or from a sentinel crossing emitted by Phase A. Punt to Phase 1 implementation.
- Whether to express Phase A's emission via a template parameter on `modelType` (compile-time) or an enum branch (runtime). Compile-time is cleaner; we can do either.